Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks, K5)

All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

Solution selected: the Q6 triage queue (`PriorityQueue<Patient>` with a comparator).

A. Diagnose the bottleneck (2 marks)

Evidence comes from the Q6 experiments.

Question	Finding from my evidence
Where is most time/memory spent?	**Showing the waiting list.** The heap is only partially ordered (Q6 §3: printing the queue gave `Zoya(S4)` before `Ravi(S3)`), so each display must copy the queue into a list and **sort it again**. For n = 100,000 waiting patients, 200 displays took **2,119 ms** (≈ 10.6 ms per display), while 100,000 admit+serve operations took only 36 ms.
Which operation grows significantly with data size?	the display (copy + sort = O(n log n)): from 22 ms at n = 1,000 to 2,119 ms at n = 100,000 for 200 displays (×96 for ×100 n, i.e. n log n). **Aging** has the same problem: `PriorityQueue` cannot change the priority of an element in place, so the fair rule from Q6 requires polling out and re-adding **every** patient, O(n log n) per aging pass (47 ms per 20 passes at n = 100,000).
Is work unnecessarily repeated?	**Yes:** the same waiting patients are re-sorted every time the list is displayed, although their relative order did not change since the last display (re-sorting, as the assignment's example suggests). Aging rebuilds the whole heap even when only the one patient

Question	Finding from my evidence
	at the head of each level might be overdue.
Which characteristic of the chosen class causes the limitation?	`PriorityQueue` offers *only* "give me the minimum": it keeps a **partial order** (heap property), has no ordered iteration and no efficient update/remove of an arbitrary element (`remove(Object)` is O(n)). It treats priorities as an arbitrary, unbounded set, but here the priority takes only **5 distinct values**, and arrival order is already the natural order inside a level, so a general-purpose heap does more work than the problem needs.

B. Design objective (2 marks)

The redesign must (measurable targets):

1. **Display the waiting list in treatment order without sorting:** O(n) and **at least 10× faster** than the original for n ≥ 5,000.
2. **Serve and admit in O(1)** regardless of n (no growth with queue size).
3. **Implement the aging rule without scanning or rebuilding the queue:** O(1) per decision.
4. **Preserve correctness of ordering:** with aging disabled, the service order and the waiting list must be *identical* to the original design for any sequence of operations (checked on 200,000 random operations).
5. Guarantee that a severity-5 patient is served within the configured limit (e.g. 240 ticks) even under overload.

C. Revised solution (4 marks)

What I changed: replaced the single `PriorityQueue` by **five FIFO queues** (`ArrayDeque`), one per severity level:

```
class LevelTriage
    queues[1..5] : ArrayDeque<Patient>                          // one
FIFO queue per severity
    admit(severity):  queues[severity].addLast(patient)
O(1)
    serve(now, limit[1..5]):
        overdue ← none
        FOR s = 1 TO 5 DO
// only 5 heads are inspected
            h ← queues[s].peekFirst()
```

```
            IF h ≠ null AND now − h.arrival > limit[s] AND (overdue =
none OR h.arrival < overdue.arrival) THEN overdue ← h
        IF overdue ≠ none THEN RETURN
queues[overdue.severity].pollFirst()   // starvation guard
        FOR s = 1 TO 5 DO IF queues[s] not empty THEN RETURN
queues[s].pollFirst()
        RETURN null
    waitingList():  concatenate queues[1], queues[2], ..., queues[5]
O(n), already in order
```

(Class diagram: `LevelTriage` has-5 `ArrayDeque<Patient>`; `Patient(id, severity, arrival)`.)

Question	Answer
Which property do I exploit?	(1) only **5** possible priorities; (2) patients are added in arrival order, so each queue is **already sorted by arrival**; the second sort key (arrival counter) is not needed at all. (3) Inside a level, only the **head** can be the longest-waiting patient, so checking whether a level has an overdue patient means checking one element.
Which operations are avoided or reduced?	the copy + sort for display (now a plain concatenation, O(n)); the full heap rebuild for aging (now 5 head checks per serve, O(1)); the O(log n) sift-up/down per admit and serve.
Additional memory or preprocessing?	5 queue objects instead of 1 heap; each `ArrayDeque` keeps spare capacity (up to 2× in its array), so memory is O(n) with a slightly larger constant. No preprocessing.
New trade-offs	(i) works **only because the number of levels is small and fixed**: with continuous priorities (e.g. a score from 0–1000, or many levels) the per-level approach does not scale and the heap is the right choice; (ii) the comparator can no longer be changed freely: the "two-level" rule is now built into the data structure; (iii) the waiting list shows the *normal* order, and overdue patients, who will be served before it, are not yet marked in the list (the display can flag them

Question	Answer
	using the same check); (iv) the aging limits are a medical policy choice.

Source (the original and the redesign together with the experiment):

```
import java.util.*;

/** ORIGINAL (from Q6): one PriorityQueue. Needs a copy + sort to show
the waiting list; aging = rebuild. */
class OriginalTriage {
    record P(int id, int sev, long arr) {}
    static final Comparator<P> ORDER =
Comparator.comparingInt(P::sev).thenComparingLong(P::arr);
    final PriorityQueue<P> pq = new PriorityQueue<>(ORDER);
    long counter = 0;
    void admit(int sev) { pq.add(new P((int) counter, sev, counter+
+)); }
    P serve() { return pq.poll(); }
    List<P> waitingList() { List<P> l = new ArrayList<>(pq);
l.sort(ORDER); return l; }     // copy + sort EVERY display
    /** Aging: anyone who has waited longer than 'limit' ticks gets
more urgent by one level -> must rebuild the heap. */
    void age(long now, long limit) {
        List<P> all = new ArrayList<>(pq); pq.clear();
        for (P p : all) pq.add(now - p.arr() > limit && p.sev() > 1 ?
new P(p.id(), p.sev() - 1, p.arr()) : p);   // O(n log n)
    }
}

/** REDESIGN: one FIFO queue per severity level (only 5 levels, and
arrival order is already queue order). */
class LevelTriage {
    record P(int id, int sev, long arr) {}
    static final int LEVELS = 5;
    @SuppressWarnings("unchecked") final ArrayDeque<P>[] q = new
ArrayDeque[LEVELS + 1];
    long counter = 0; int size = 0;
    LevelTriage() { for (int i = 1; i <= LEVELS; i++) q[i] = new
ArrayDeque<>(); }
    void admit(int sev) { q[sev].addLast(new P((int) counter, sev,
counter++)); size++; }       // O(1)
    /** Serve: any patient whose wait exceeds limit[sev] is served
first (oldest overdue); otherwise most severe first. O(5). */
    P serve(long now, long[] limit) {
        P overdue = null;
        for (int s = 1; s <= LEVELS; s++) { P h = q[s].peekFirst(); if
(h != null && now - h.arr() > limit[s] && (overdue == null || h.arr()
< overdue.arr())) overdue = h; }
        if (overdue != null) { size--; return
```

```
q[overdue.sev()].pollFirst(); }
        for (int s = 1; s <= LEVELS; s++) if (!q[s].isEmpty())
{ size--; return q[s].pollFirst(); }
        return null;
    }
    P serve() { return serve(0, new long[]{0, Long.MAX_VALUE,
Long.MAX_VALUE, Long.MAX_VALUE, Long.MAX_VALUE, Long.MAX_VALUE}); }
// no aging
    List<P> waitingList() { List<P> l = new ArrayList<>(size); for
(int s = 1; s <= LEVELS; s++) l.addAll(q[s]); return l; }   // already
in order: O(n), no sort
}

public class Triage {
    static double ms(Runnable r) { long t = System.nanoTime();
r.run(); return (System.nanoTime() - t) / 1e6; }

    public static void main(String[] args) {
        // ---------- correctness: both designs must serve in the SAME
order with no aging
        Random r = new Random(11); OriginalTriage o = new
OriginalTriage(); LevelTriage l = new LevelTriage(); boolean same =
true; int served = 0;
        for (int i = 0; i < 200_000; i++) {
            if (r.nextInt(3) < 2) { int s = 1 + r.nextInt(5);
o.admit(s); l.admit(s); }
            else { var a = o.serve(); var b = l.serve(); if ((a ==
null) != (b == null) || (a != null && a.id() != b.id())) same = false;
served++; }
        }
        boolean listSame = true; var wl1 = o.waitingList(); var wl2 =
l.waitingList();
        if (wl1.size() != wl2.size()) listSame = false; else for (int
i = 0; i < wl1.size(); i++) if (wl1.get(i).id() != wl2.get(i).id())
listSame = false;
        System.out.println("Correctness (200,000 random operations, no
aging): same service order = " + same + ", same waiting list = " +
listSame + " (" + wl1.size() + " waiting)");

        // ---------- performance: display the waiting list 200 times,
admit+serve, and aging pass
        System.out.printf("%n%8s | %-34s | %-34s%n", "waiting",
"ORIGINAL (PriorityQueue)", "REDESIGN (5 FIFO queues)");
        System.out.printf("%8s | %10s %10s %10s | %10s %10s %10s
(ms; display = 200 displays, ops = 100k admit+serve, aging = 20
passes)%n", "n", "display", "ops", "aging", "display", "ops",
"aging");
        for (int n : new int[]{1_000, 5_000, 20_000, 100_000}) {
            double[] res = new double[6]; int reps = 3;
            for (int rep = 0; rep < reps; rep++) {
```

```
                OriginalTriage a = new OriginalTriage(); LevelTriage b
= new LevelTriage(); Random rr = new Random(5);
                for (int i = 0; i < n; i++) { int s = 1 +
rr.nextInt(5); a.admit(s); b.admit(s); }
                res[0] += ms(() -> { int k = 0; for (int i = 0; i <
200; i++) k += a.waitingList().size(); if (k < 0)
System.out.println(); });
                res[3] += ms(() -> { int k = 0; for (int i = 0; i <
200; i++) k += b.waitingList().size(); if (k < 0)
System.out.println(); });
                res[1] += ms(() -> { Random q = new Random(2); for
(int i = 0; i < 100_000; i++) { a.admit(1 + q.nextInt(5)); a.serve();
} });
                res[4] += ms(() -> { Random q = new Random(2); long[]
lim = {0, 1_000_000, 1_000_000, 1_000_000, 1_000_000, 1_000_000}; for
(int i = 0; i < 100_000; i++) { b.admit(1 + q.nextInt(5)); b.serve(i,
lim); } });
                res[2] += ms(() -> { for (int i = 0; i < 20; i++)
a.age(a.counter, n); });        // rebuild every pass
                res[5] += ms(() -> { long[] lim = {0, 1000, 1000,
1000, 1000, 1000}; for (int i = 0; i < 20; i++) { b.admit(3);
b.serve(b.counter, lim); } });   // aging is inside serve(): 20 serves
            }
            System.out.printf("%8d | %10.1f %10.1f %10.1f | %10.1f
%10.1f %10.1f%n", n, res[0] / reps, res[1] / reps, res[2] / reps,
res[3] / reps, res[4] / reps, res[5] / reps);
        }

        // ---------- fairness: sev-5 patient under overload (3
arrivals of sev 1-4 per tick, 1 served per tick)
        long[] limit = {0, 0, 30, 60, 120, 240};   // max wait (ticks)
tolerated per severity; sev-1 never overdue first
        limit[1] = Long.MAX_VALUE;
        LevelTriage f = new LevelTriage(); f.admit(5); long waited = -
1; Random rr = new Random(3);
        for (int t = 1; t <= 5000 && waited < 0; t++) {
            for (int k = 0; k < 3; k++) f.admit(1 + rr.nextInt(4));
            var p = f.serve(t, limit); if (p.sev() == 5) waited = t;
        }
        System.out.println("\nFairness: severity-5 patient (limit 240
ticks) served after " + waited + " ticks under overload (original
design: never, see Q6).");
    }
}
```

D. Prediction before implementing (1 mark)

(Written from complexity analysis. For your own submission, write and date your own prediction before running the program.)

- **Display:** the original costs O(n log n) per display, the redesign O(n): I expect the improvement to **grow with n**, roughly by a factor of $\log_2 n$ (≈ 10–17) as a theoretical minimum, and more in practice because sorting objects through a comparator is slower than copying: **≥ 10× faster for n ≥ 5,000**.
- **Admit + serve:** O(log n) vs O(1): a modest improvement, **about 2–10×**, growing slowly with n.
- **Aging:** original O(n log n) per pass versus O(1) per decision: from negligible at n = 1,000 to very large at n = 100,000 (the two numbers are not the same unit; see the table note).
- **Correctness:** identical service order and waiting list when aging is off.

E. Implement and validate (2 marks)

Correctness (same test conditions): 200,000 random operations (about 2/3 admits, 1/3 serves, severities 1–5): **same service order: true; same waiting list: true** (66,286 patients waiting at the end).

Performance (`ms`; display = 200 displays of the whole waiting list; ops = 100,000 × (admit + serve); aging: original = 20 full aging passes, redesign = 20 `serve` calls that include the overdue check; average of 3 runs):

Waiting patients n	Original: display	Original: ops	Original: aging	Redesign: display	Redesign: ops	Redesign: aging/serve
1,000	22.1	17.1	2.2	1.3	5.2	0.4
5,000	87.9	14.8	3.5	4.2	3.9	0.0
20,000	376.1	20.9	5.4	6.4	2.9	0.0
100,000	2,119.4	36.1	47.1	33.5	3.2	0.1

n	Display speed-up	Admit+serve speed-up
1,000	17×	3.3×
5,000	21×	3.8×
20,000	59×	7.2×
100,000	63×	11.3×

Fairness test: under overload (3 arrivals of severity 1–4 per tick, 1 patient served per tick), a severity-5 patient with a limit of 240 ticks was served after **241 ticks**; the original design never served the patient in 20,000 ticks (Q6).

F. Defend the final design (2 marks)

- **Was the prediction supported?** Yes. Display improved 17–63× (≥ 10× as predicted, increasing with n: ×17 at 1,000 to ×63 at 100,000), admit+serve improved 3–11× (predicted 2–10×; the largest value is slightly above my range), and the correctness test passed.

- **Magnitude of improvement:** for a screen that refreshes the waiting list, the cost per display at n = 100,000 fell from about 10.6 ms to 0.17 ms. For realistic emergency-room sizes (hundreds of patients) both are already far too fast to matter (about 0.1 ms per display for the original at n = 1,000), so the practical gain is **mostly the aging rule**: the original cannot implement the fair rule without a full rebuild, whereas the redesign does it in O(1) and demonstrably prevents starvation.
- **New costs / trade-offs:** restricted to a few, fixed levels; the policy limits must be configured; the comparator rule is hard-wired; slightly more memory for 5 queues; `ArrayDeque` is not thread-safe (as for `PriorityQueue`).
- **When the original is still preferable:** when the priority is a **continuous or large range** (many levels or a computed score), when the queue is **small** and the list is shown rarely (the redesign brings no measurable benefit and the heap is simpler, shorter code), when the ordering rule may change often (`Comparator` is one line to change), or when you need only "next patient" and never the full ordered list.
- **Conclusion:** the redesign is justified because it removes the two *repeated* costs diagnosed in A (re-sorting for display and re-building for aging) by using what the problem actually guarantees (5 levels, arrival order), not simply by replacing one collection with a faster one; the improvement is measured, and the behaviour with aging disabled is proved identical.